Dynamic Programming

Introduction

- Greedy algorithms that make the best local decision at each step are typically efficient, but usually do not guarantee global optimality. Exhaustive search algorithms that try all possibilities and select the best always produce the optimum result, but usually at a prohibitive cost in terms of time complexity.

- Dynamic programming gives us a way to design custom algorithms which systematically search all possibilities (thus guaranteeing correctness) while storing results to avoid recomputing (thus providing efficiency).

Recurrent Relations

- A recurrence relation is an equation which is defined in terms of itself. They are useful because many natural functions are easily expressed as recurrences:

  Polynomials: $a_n = a_{n-1} + 1; a_1 = 1 \rightarrow a_n = n$

  Exponentials: $a_n = 2a_{n-1}; a_1 = 2 \rightarrow a_n = 2^n$

  Factorial: $a_n = na_{n-1}; a_1 = 1 \rightarrow a_n = n!$

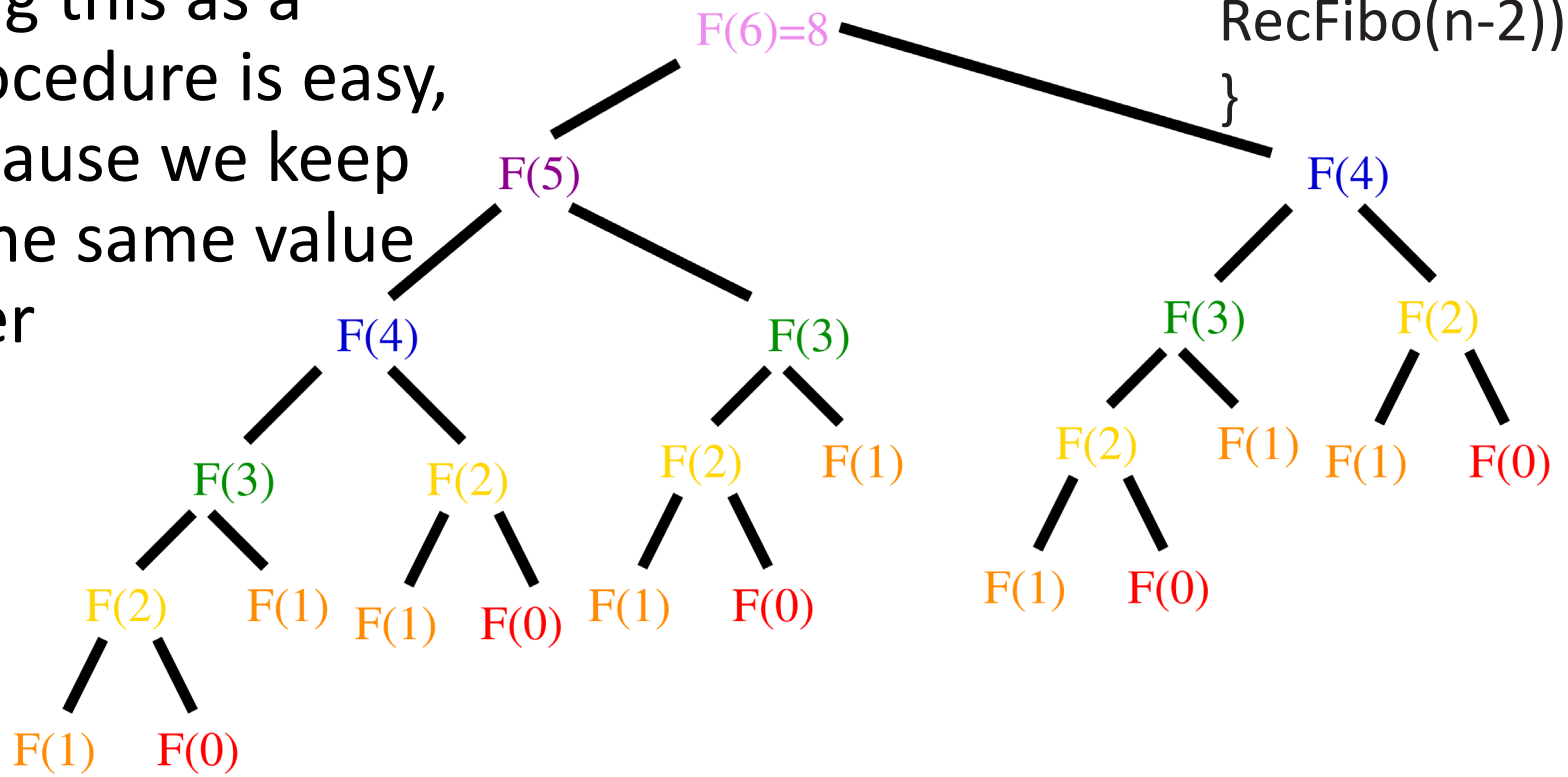
- Computer programs can easily evaluate the value of a given recurrence even without the existence of a nice closed form

Computing Fibonacci Numbers

$F_n = F_{n-1} + F_{n-2}$; $F_0 = 0$; $F_1 = 1$

Implementing this as a recursive procedure is easy, but slow because we keep calculating the same value over and over

```
long RecFibo(int n) {
if (n == 0) {
return(0);
}
if (n == 1) {
return(1);
}
return(RecFibo(n-1) +
RecFibo(n-2));
}
```



Memo(r)ization

- The obvious reason for the recursive algorithm's lack of speed is that it computes the same Fibonacci numbers over and over and over

- A single call to RecFibo(n) results in one recursive call to RecFibo(n − 1), two recursive calls to RecFibo(n − 2), three recursive calls to RecFibo(n − 3), five recursive calls to RecFibo(n − 4) etc.

- In general F_{k-1} recursive calls to RecFibo(n − k) for any integer $0 \leq k <$ n. Each call is recomputing some Fibonacci number from scratch

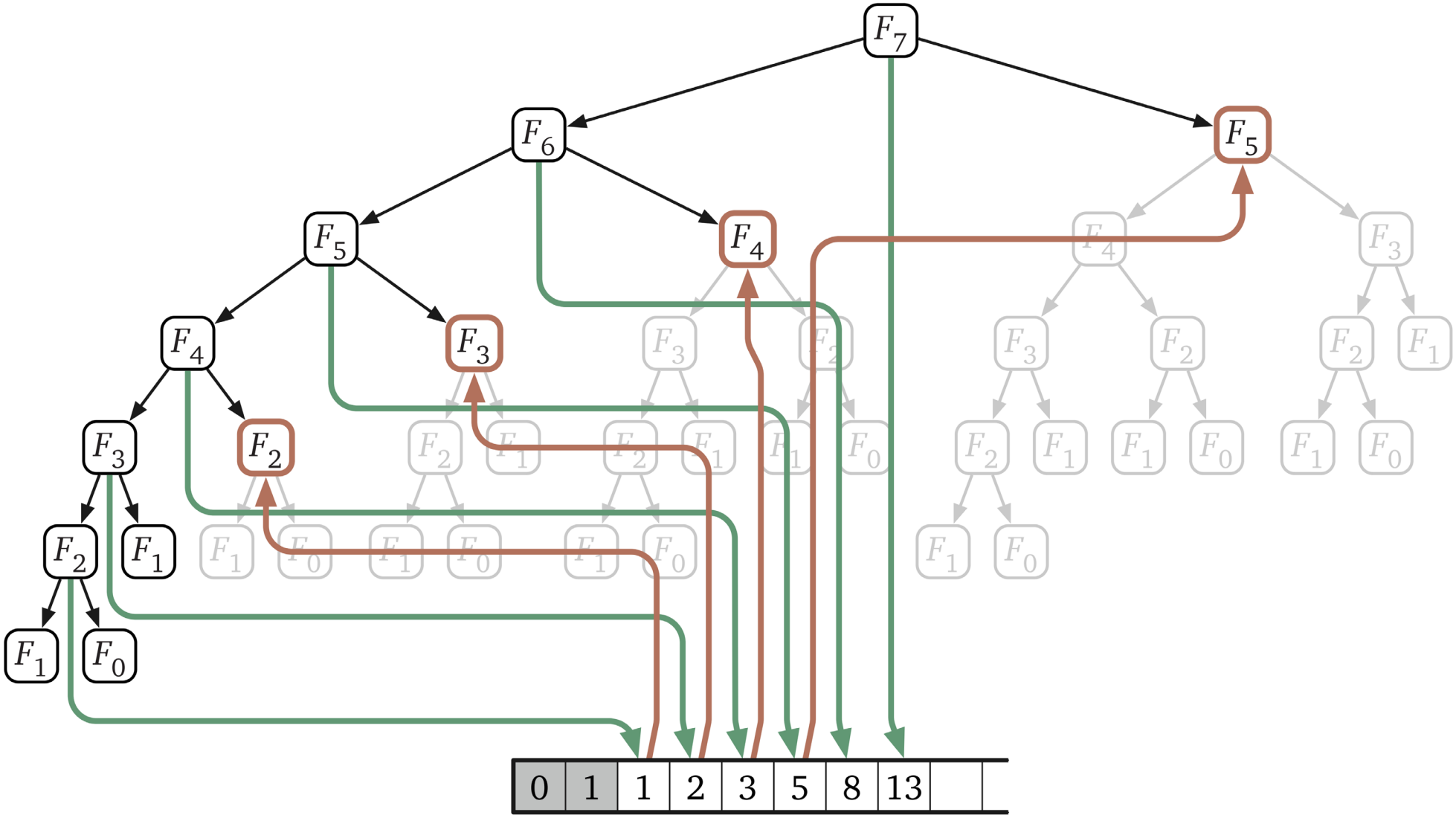
Memo(r)ization

- We can speed up our recursive algorithm considerably by writing down the results of our recursive calls and looking them up again if we need them later

- This optimization technique is known as memoization

MemFibo(*n*):
```
if n = 0
  return 0
else if n = 1
  return 1
else
  if F[n] is undefined
    F[n] ← MemFibo(n - 1) + MemFibo(n - 2)
  return F[n]
```

Improved Recursion



Dynamic Programming

- We can replace the memoized recurrence with a simple for-loop that intentionally fills the array in that order, instead of relying on a more complicated recursive algorithm to do it for us accidentally

IterFibo(n):

```
F[0] =  0
F[1] = 1
for i = 2 to n
        F[i] = F[i − 1] + F[i − 2]
return F[n]
```

Don’t Remember All

```
IterFibo2(n):
        prev = 0
        curr = 1
        for i = 1 to n
                next = curr + prev
                prev = curr
                curr = next
        return curr
```

In a nutshell, dynamic programming is *recursion without repetition*

Edit distance

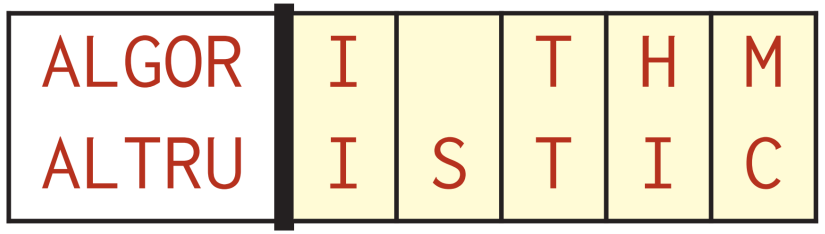
- The edit distance between two strings is the minimum number of letter insertions, letter deletions, and letter substitutions required to transform one string into the other. For example, the edit distance between FOOD and MONEY is at most four:

  FOOD → MOOD → MON D → MONED → MONEY

- We can visualize this editing process by aligning the strings one above the other, with a gap in the first word for each insertion and a gap in the second word for each deletion. Columns with two different characters correspond to substitutions

Recursive Structure

- To develop a dynamic programming algorithm to compute edit distance, we first need to formulate the problem recursively





Because the cost of an alignment is just the number of mismatched columns, our remaining decisions don't depend on the editing operations we've already chosen; they only depend on the prefixes we haven't aligned yet

Recursive Structure

- For any two input strings A[1.. m] and B[1.. n], we can formulate the edit distance problem recursively as follows:
  - For any indices i and j, let Edit(i, j) denote the edit distance between the prefixes A[1.. i] and B[1.. j]. We need to compute Edit(m, n)
- When i and j are both positive, there are exactly three possibilities for the last column in the optimal alignment of A[1 .. i] and B[1.. j]
  - Insertion: The last entry in the top row is empty. In this case, the edit distance is equal to Edit(i, j −1) +1. The +1 is the cost of the final insertion and the recursive expression gives the minimum cost for the remaining alignment



Recursive Structure



- Deletion: The last entry in the bottom row is empty. In this case, the edit distance is equal to Edit(i − 1, j) + 1. The +1 is the cost of the final deletion, and the recursive expression gives the minimum cost for the remaining alignment

- Substitution: Both rows have characters in the last column. If these two characters are different, then the edit distance is equal to Edit(i−1, j−1)+1. If these two characters are equal, the substitution is free, so the edit distance is Edit(i − 1, j − 1)





Recurrence

- This generic case analysis breaks down if either i = 0 or j = 0, but those boundary cases are easy to handle directly.
  - Transforming the empty string into a string of length j requires j insertions, so Edit(0, j) = j.
  - Transforming a string of length i into the empty string requires i deletions, so Edit(i, 0) = i.

$$Edit(i,j) = \begin{cases} i & \text{if } j = 0 \\ j & \text{if } i = 0 \\ \min \begin{Bmatrix} Edit(i, j-1) + 1 \\ Edit(i-1, j) + 1 \\ Edit(i-1, j-1) + [A[i] \neq B[j]] \end{Bmatrix} & \text{otherwise} \end{cases}$$

Insertion: Intuition

- Suppose A="cat" and B="cats". If you want to compute Edit(3,4) (transform "cat" into "cats"):

- You can insert the character 's' into A. After the insertion, A becomes "cats", and the problem reduces to comparing "cat" with "cat".

- This is equivalent to solving Edit(3,3), plus the cost of the insertion.

Deletion: Intuition

- Suppose A="cats" and B="cat". If you want to compute Edit(4,3) (transform "cats" into "cat"):

- You can delete the character 's’ from A. After the deletion, A becomes "cat", and the problem reduces to comparing "cat" with "cat".

- This is equivalent to solving Edit(3,3), plus the cost of the deletion.

Substitution: Intuition

- Suppose A="cat" and B="cut". If you want to compute Edit(3,3) (transform "cat" into "cut"):

- You compare the last characters of the current substrings: A[3]=’t’ and B[3]=’t’.
  - If A[3]=B[3], no substitution is needed, and the cost is 0. So, you just solve Edit(2,2).
  - If A[3]≠B[3], you substitute A[3] with B[3], incurring a cost of 1. Then, you solve Edit(2,2)

Dynamic Programming

- Memoization structure: So we can memoize all possible values of Edit(i, j) in a two-dimensional array Edit[0 .. m, 0.. n]
- Dependencies: Each entry Edit[i, j] depends only on its three neighboring entries Edit[i – 1, j], Edit[i, j – 1], and Edit[i – 1, j – 1]

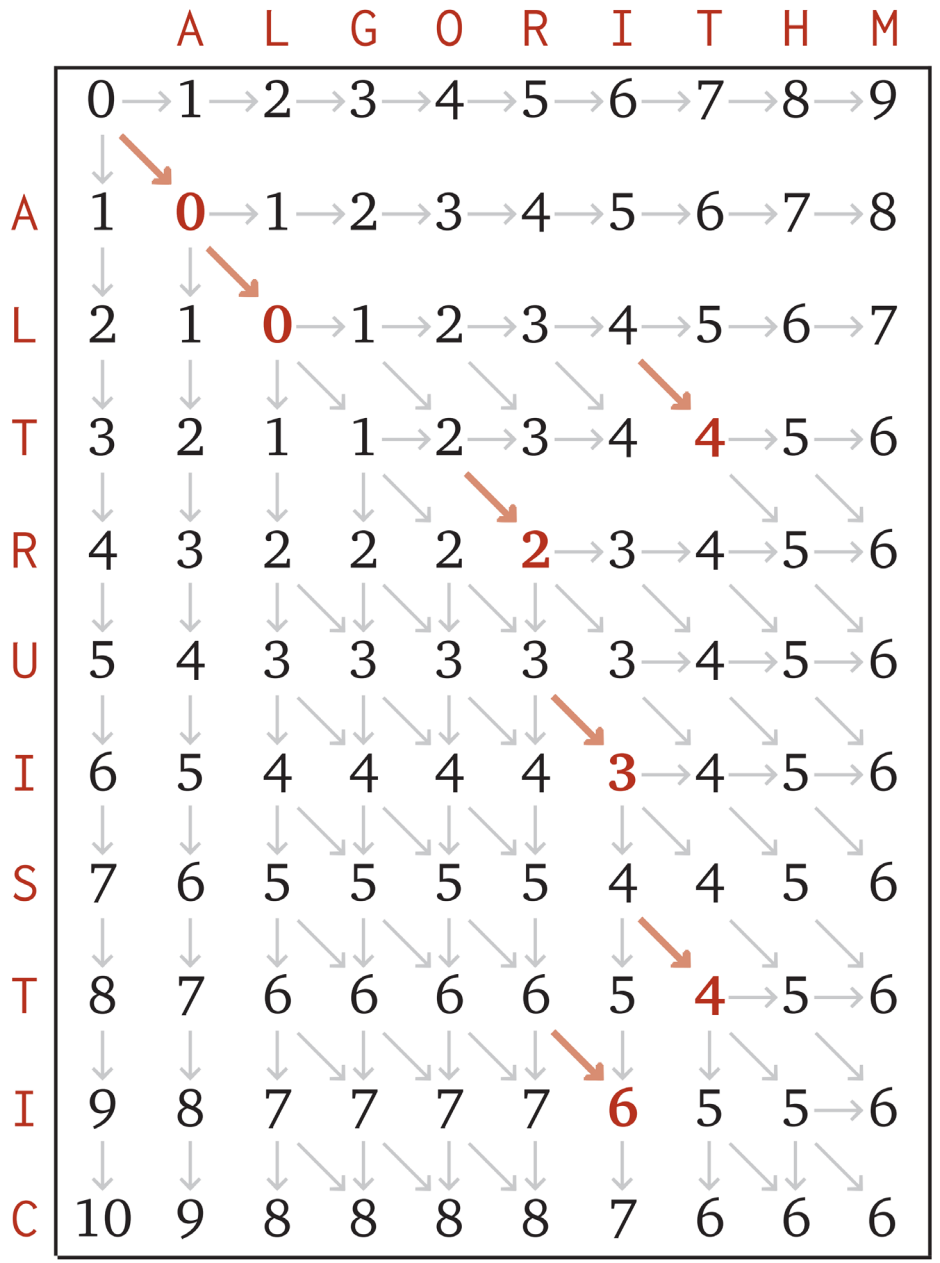
```
EditDistance(A[1..m], B[1..n]):
    for j ← 0 to n
        Edit[0, j] ← j

    for i ← 1 to m
        Edit[i, 0] ← i
        for j ← 1 to n
            ins ← Edit[i, j − 1] + 1
            del ← Edit[i − 1, j] + 1
            if A[i] = B[j]
                rep ← Edit[i − 1, j − 1]
            else
                rep ← Edit[i − 1, j − 1] + 1
            Edit[i, j] ← min {ins, del, rep}

    return Edit[m, n]
```

Memoization Table

- Bold numbers indicate places where characters in the two strings are equal
- The edit distance between **ALGORITHM** and **ALTRUISTIC** is six (bottom right)
- Each direction of arrow corresponds to a different edit operation: horizontal=deletion, vertical=insertion, and diagonal=substitution. Bold red diagonal arrows indicate “free” substitutions of a letter for itself.



Solution

```
A L G O R I   T H M
A L T R U I S T I C

A L G O R   I   T H M
A L   T R U I S T I C

A L G O R   I   T H M
A L T   R U I S T I C
```

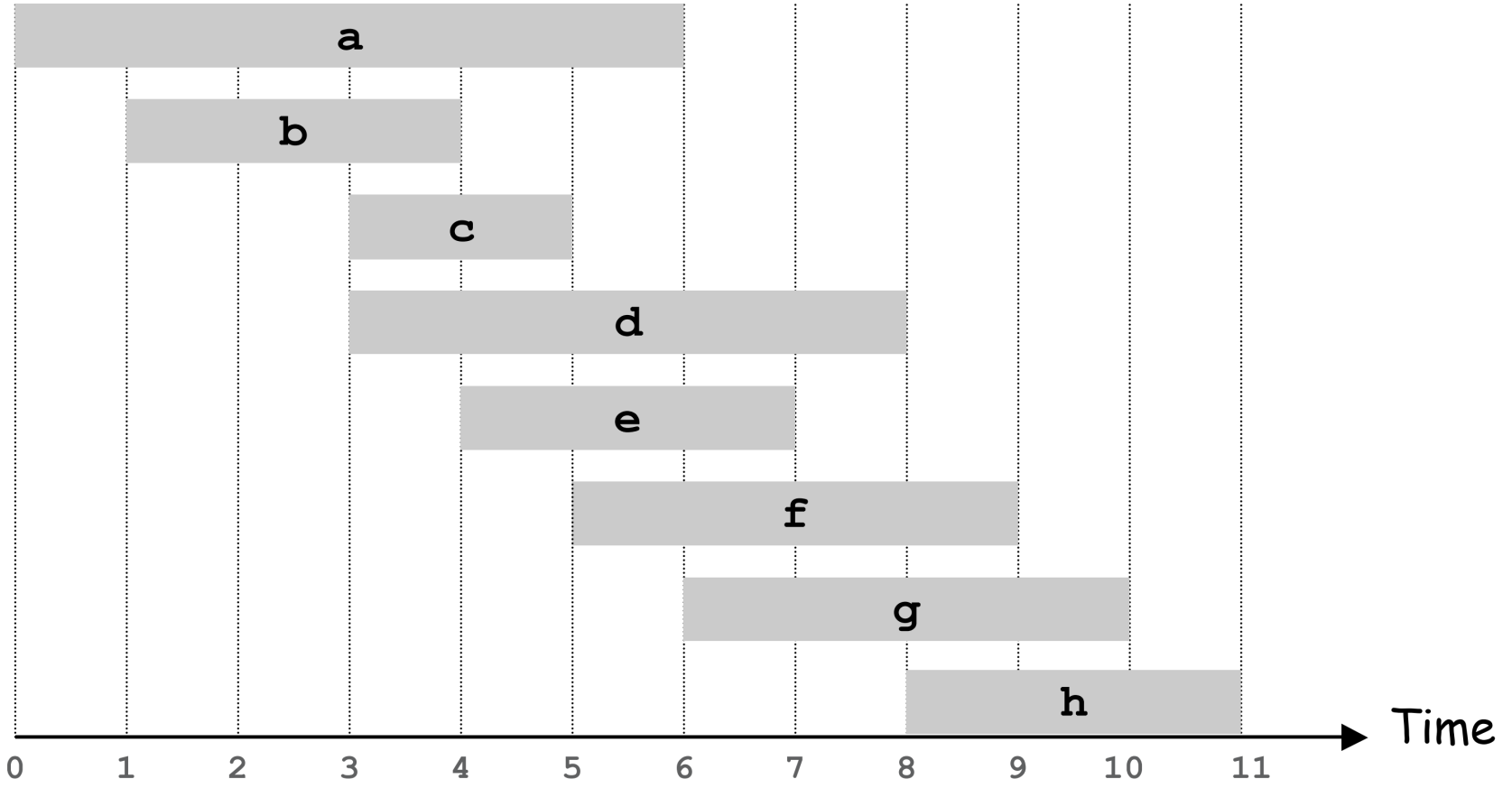
Interval Scheduling

Consider the following very simple scheduling problem. You have a resource— it may be a lecture room, a supercomputer, or an electron microscope—and many people request to use the resource for periods of time. A request takes the form: Can I reserve the resource starting at time s, until time f?

We will assume that the resource can be used by at most one person at a time. A scheduler wants to accept a subset of these requests, rejecting all others, so that the accepted requests do not overlap in time. The goal is to maximize the number of requests accepted.

Interval Scheduling

- Job j starts at s_j and finishes at f_j.
- Two jobs compatible if they don't overlap.
- Goal: find maximum subset of mutually compatible jobs



Greedy Solution

Consider jobs in increasing order of finish time. Take each job provided it's compatible with the ones already taken

- [**Earliest start time**] Consider jobs in ascending order of s_j
- [**Earliest finish time**] Consider jobs in ascending order of f_j
- [**Shortest interval**] Consider jobs in ascending order of $f_j - s_j$
- [**Fewest conflicts**] For each job j, count the number of conflicting jobs c_j. Schedule in ascending order of c_j

Example

Job	Start (s_j)	Finish (f_j)	Interval Length ($f_j - s_j$)
1	1	4	3
2	3	5	2
3	0	6	6
4	5	7	2
5	8	9	1
6	5	9	4

[Earliest Start Time] Order: Job 3 (s3=0), Job 1 (s1=1), Job 2 (s2=3), Job 4 (s4=5), Job 6 (s6=5), Job 5 (s5=8)
Solution {Job 3, Job 5}

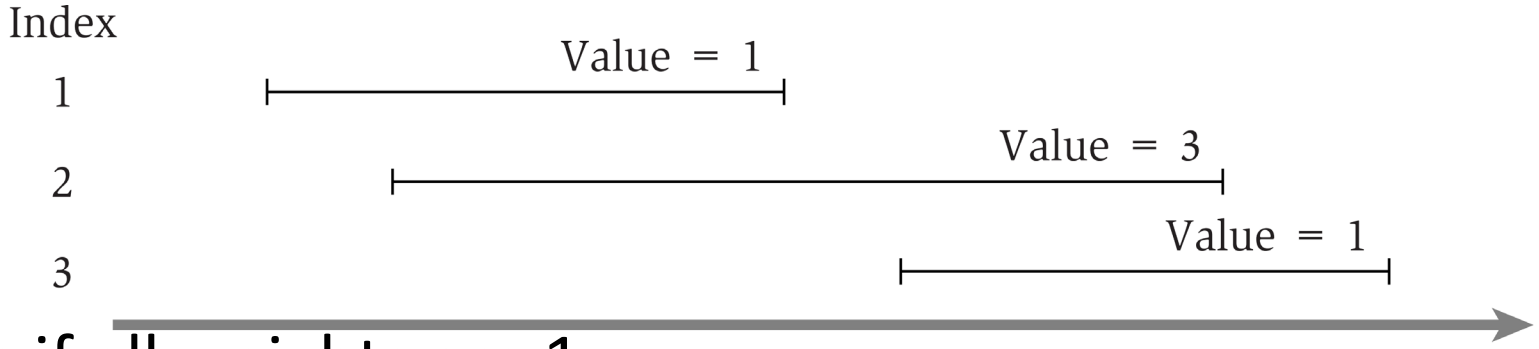
[Shortest Interval] Order: Job 5 (1), Job 2 (2), Job 4 (2), Job 1 (3), Job 6 (4), Job 3 (6)
Solution {Job 5, Job 2}

[Fewest Conflict] Order: Job 5 (c5=1), Job 6 (c6=2), Job 1 (c1=2), Job 2 (c2=2), Job 4 (c4=2), Job 3 (c3=3)
Solution {Job 5, Job 1, Job 4}

[Earliest Finish Time] Order: Job 1, Job 2, Job 3, Job 4, Job 5, Job 6
Solution {Job 1, Job 4, Job 5}

Weighted Interval Scheduling

Now, suppose more generally that each request interval i has an associated value, or weight, $v_i > 0$; we could picture this as the amount of money we will make from the ith individual if we schedule his or her request. Our goal will be to find a compatible subset of intervals of maximum total value.



Greedy algorithm works if all weights are 1.

- Consider jobs in ascending order of finish time.
- Add job to subset if it is compatible with previously chosen jobs.

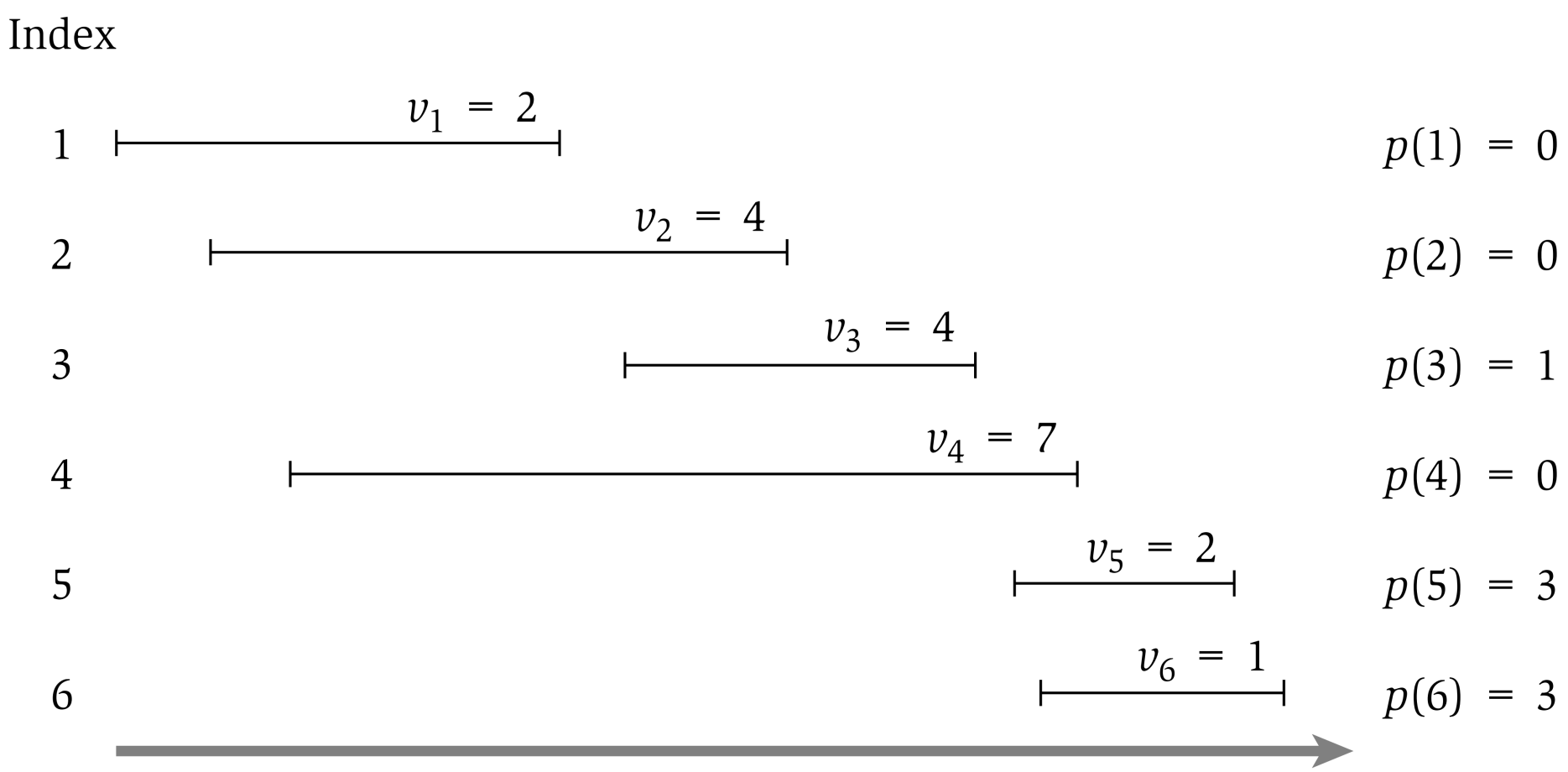
Recursive Algorithm

- Let's suppose that the requests are sorted in order of nondecreasing finish time: $f_1 \leq f_2 \leq \ldots \leq f_n$.

- We'll say a request i comes before a request j if i < j

- We define p(j), for an interval j, to be the largest index i < j such that intervals i and j are disjoint. In other words, i is the leftmost interval that ends before j begins

Recursive Algorithm



Recursive Algorithm

- OPT(j) = value of optimal solution to the problem consisting of job requests 1, 2, ..., j

- Case 1: OPT selects job j
  - collect profit v_j
  - can't use incompatible jobs { p(j) + 1, p(j) + 2, ..., j - 1 }
  - must include optimal solution to problem consisting of remaining compatible jobs 1, 2, ..., p(j)

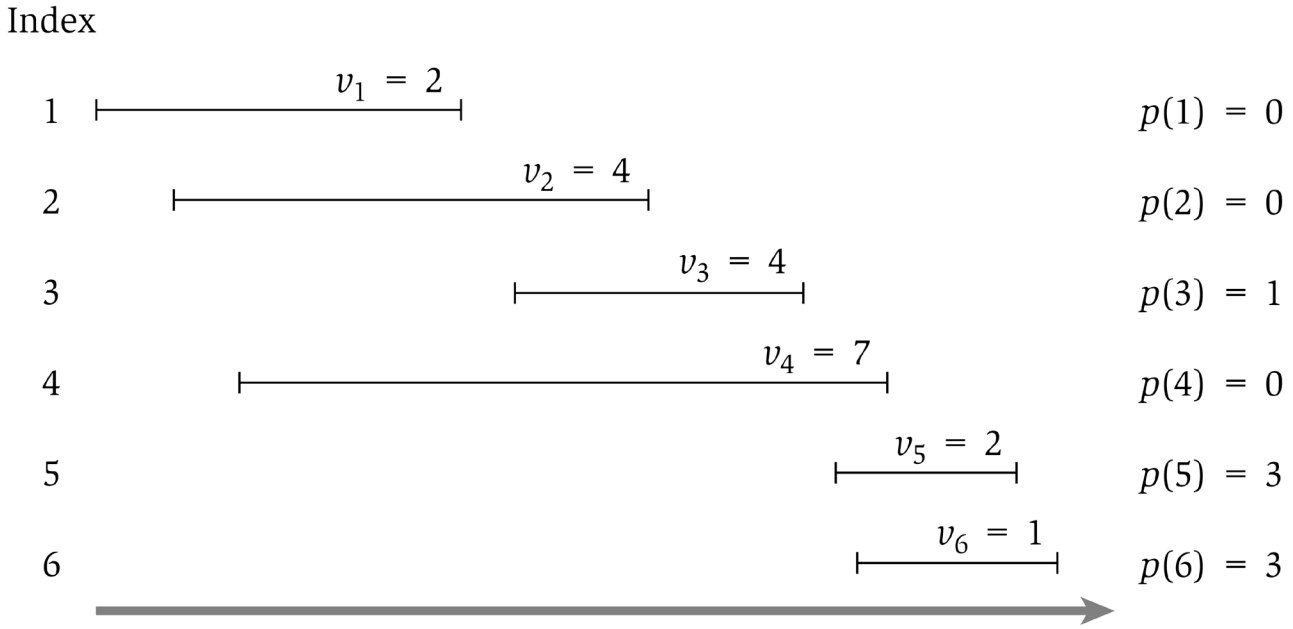
- Case 2: OPT does not select job j
  - must include optimal solution to problem consisting of remaining compatible jobs 1, 2, ..., j-1

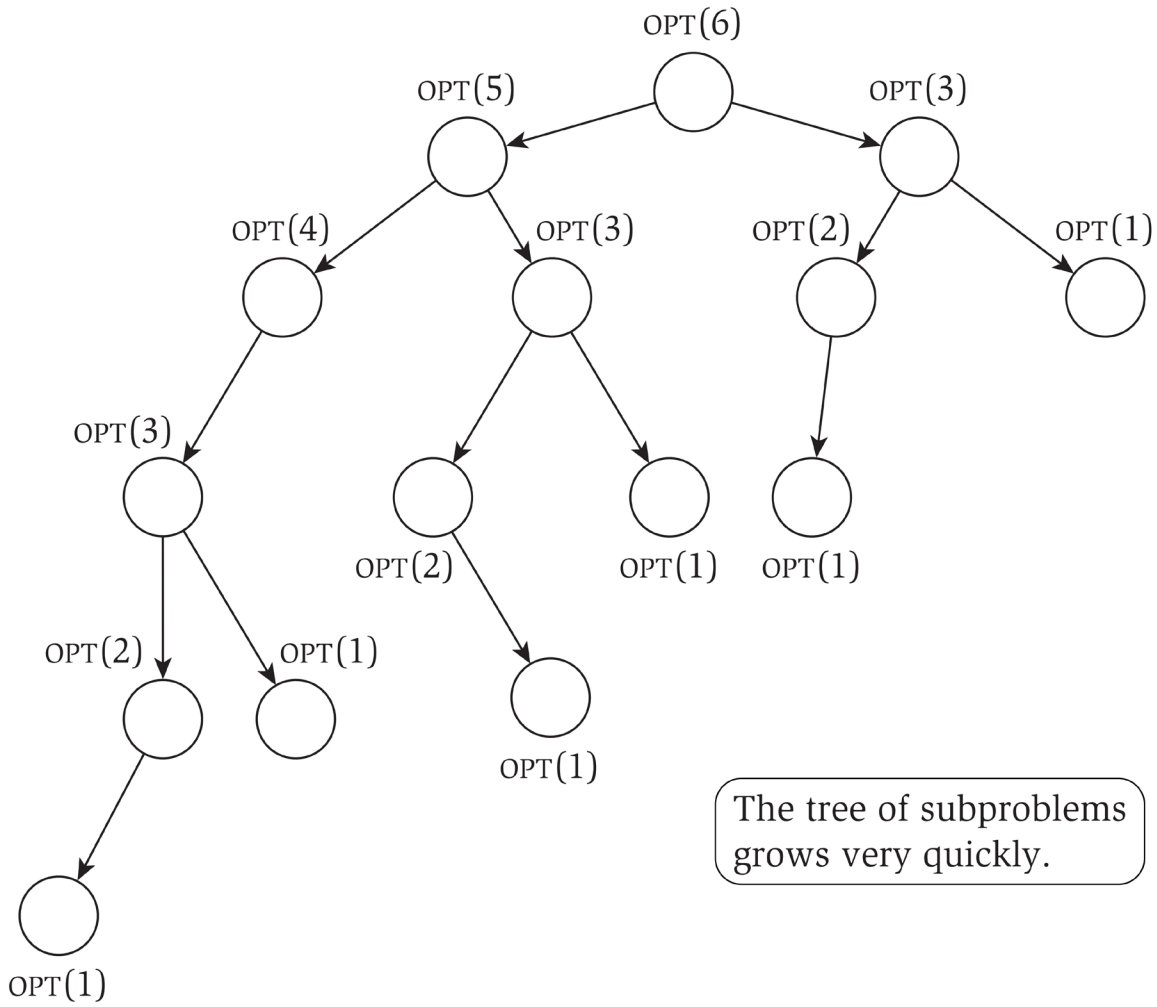
Recursive Definition

$$OPT(j) = \begin{cases} 0 & \text{if } j = 0 \\ \max\left\{ v_j + OPT(p(j)),\ OPT(j-1) \right\} & \text{otherwise} \end{cases}$$

```
Compute-Opt(j)
  If j = 0 then
    Return 0
  Else
    Return max(v_j+Compute-Opt(p(j)), Compute-Opt(j − 1))
  Endif
```

Recursive Definition





The tree of subproblems grows very quickly.

Memoization

- **Memoization**. Store results of each sub-problem in a cache; lookup as needed

```
Input: n, s1,…,sn , f1,…,fn , v1,…,vn

Sort jobs by finish times so that f1 ≤ f2 ≤ ... ≤ fn.
Compute p(1), p(2), …, p(n)

for j = 1 to n
   M[j] = empty
M[0] = 0

M-Compute-Opt(j) {
   if (M[j] is empty)
      M[j] = max(vj + M-Compute-Opt(p(j)), M-Compute-Opt(j-1))
   return M[j]
}
```

global array

Dynamic Programming

Input: $n, s_1, \ldots, s_n, f_1, \ldots, f_n, v_1, \ldots, v_n$

Sort jobs by finish times so that $f_1 \leq f_2 \leq \ldots \leq f_n$

Compute p(1), p(2), …, p(n)

```
Iterative-Compute-Opt {
     M[0] = 0
     for j = 1 to n
          M[j] = max(v_j + M[p(j)], M[j-1])
}
```

Scheduling to Minimize Lateness

Consider again a situation in which we have a single resource and a set of n requests to use the resource for an interval of time. Assume that the resource is available starting at time s. In contrast to the previous problem, however, each request is now more flexible. Instead of a start time and finish time, the request i has a deadline d_i, and it requires a contiguous time interval of length t_i, but it is willing to be scheduled at any time before the deadline. Each accepted request must be assigned an interval of time of length t_i, and different requests must be assigned nonoverlapping intervals

Lateness

- We will assign each request i an interval of time of length t_i; let us denote this interval by [s(i), f(i)], with f(i) = s(i) + t_i. Unlike the previous problem, then, the algorithm must actually determine a start time (and hence a finish time) for each interval

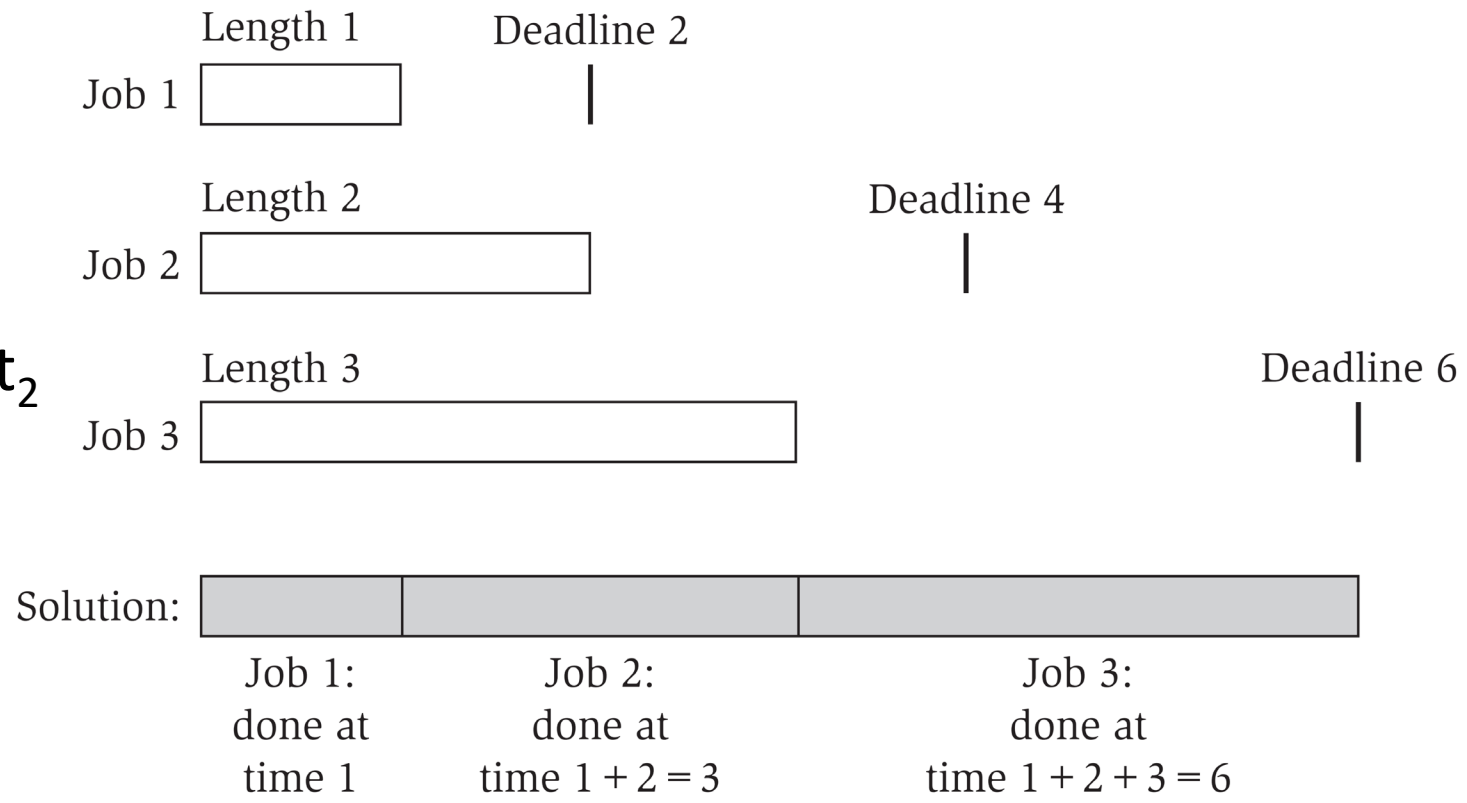
- We say that a request i is late if it misses the deadline, that is, if f(i) > d_i
  - The lateness of such a request i is defined to be $l_i = f(i) - d_i$
  - $l_i = 0$ if request i is not late

Lateness

Figure shows a sample instance of this problem, consisting of three jobs:

the first has length $t_1 = 1$ and deadline $d_1 = 2$; the second has $t_2 = 2$ and $d_2 = 4$; and the third has $t_3 = 3$ and $d_3 = 6$.

Scheduling the jobs in the order 1, 2, 3 incurs a maximum lateness of 0



Greedy Algorithm

- **[Shortest processing time first]** Consider jobs in ascending order of processing time t_j
- **[Earliest deadline first]** Consider jobs in ascending order of deadline d_j
- **[Smallest slack]** Consider jobs in ascending order of slack $d_j - t_j$

Counter Examples

- Shortest Processing Time First

	1	2
t_j	1	10
d_j	100	10

- Smallest Slack

	1	2
t_j	1	10
d_j	2	10

Earliest Deadline First

```
Sort n jobs by deadline so that d_1 ≤ d_2 ≤ … ≤ d_n

t ← 0
for j = 1 to n
   Assign job j to interval [t, t + t_j]
   s_j ← t, f_j ← t + t_j
   t ← t + t_j
output intervals [s_j, f_j]
```

Huffman Codes and Data Compression

- **Encoding Symbols Using Bits** Since computers ultimately operate on sequences of bits, one needs encoding schemes that take text written in richer alphabets (such as the alphabets underpinning human languages) and converts this text into long strings of bits

- The simplest way to do this would be to use a **fixed number of bits for each symbol** in the alphabet, and then just concatenate the bit strings for each symbol to form the text

- To take a basic example, suppose we wanted to encode the 26 letters of English, plus the space (to separate words) and five punctuation characters: comma, period, question mark, exclamation point, and apostrophe. This would give us 32 symbols in total to be encoded. Now, you can form 2^b different sequences out of b bits, and so if we use 5 bits per symbol, then we can encode $2^5 = 32$ symbols—just enough for our purposes

Average Bits and Data Compression

- In English, for example, the letters e, t, a, o, i, and n get used much more frequently than q, j, x, and z (by more than an order of magnitude).

- So it’s really a tremendous waste to translate them all into the same number of bits; instead we could use a small number of bits for the frequent letters, and a larger number of bits for the less frequent ones, and hope to end up using fewer than five bits per letter when we average over a long string of typical text

Prefix Codes

- Map letters to bit strings in such a way that no encoding is a prefix of any other

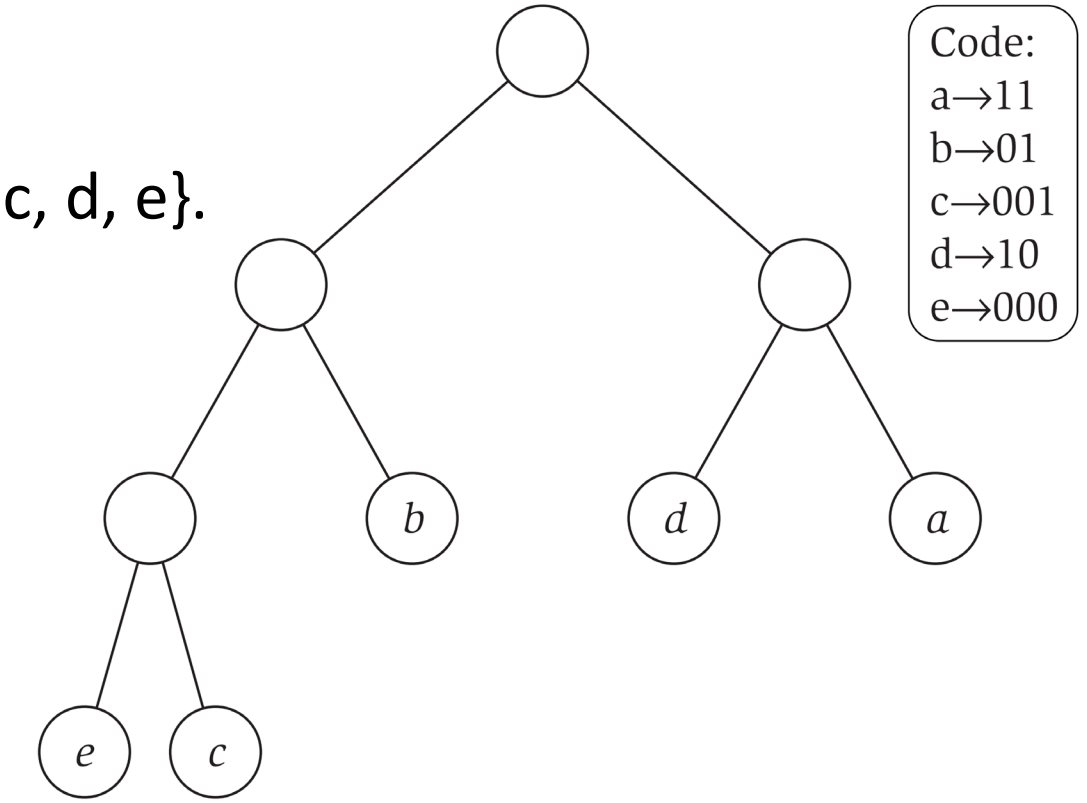
- Prefix code for a set S of letters is a function γ that maps each letter $x \in S$ to some sequence of zeros and ones, in such a way that for distinct $x, y \in S$, the sequence γ(x) is not a prefix of the sequence γ(y)

Decode:

- Scan the bit sequence from left to right
- As soon as you've seen enough bits to match the encoding of some letter, output this as the first letter of the text
- delete the corresponding set of bits from the front of the message and iterate

Prefix Codes and Trees

For example, suppose we are trying to encode the set of five letters S = {a, b, c, d, e}.
The encoding γ1 specified by
γ1(a) = 11
γ1(b) = 01
γ1(c) = 001
γ1(d) = 10
γ1(e) = 000



The string cecab would be encoded as 0010000011101

Prefix Codes and Trees

- Search for an optimal prefix code can be viewed as the search for a binary tree T, together with a labeling of the leaves of T, that minimizes the average number of bits per letter

- This average quantity has a natural interpretation in the terms of the structure of T: the length of the encoding of a letter $x \in S$ is simply the length of the path from the root to the leaf labeled x

- We refer to the length of this path as the depth of the leaf: $\text{depth}_T(v)$. Thus we are seeking the labeled tree that minimizes the weighted average of the depths of all leaves, where the average is weighted by the frequencies of the letters that label the leaves: $\sum_{x \in S} f_x \cdot \text{depth}_T(x)$

Optimal Prefix Code

- Suppose that $y*$ and $z*$ are the two lowest-frequency letters in S
  - it is safe to “lock them together” in thinking about the solution, because we know they end up as sibling leaves below a common parent
  - common parent acts like a “meta-letter” whose frequency is the sum of the frequencies of $y*$ and $z*$
  - This directly suggests an algorithm: we replace $y*$ and $z*$ with this meta-letter, obtaining an alphabet that is one letter smaller
  - We recursively find a prefix code for the smaller alphabet, and then “open up” the meta-letter back into $y*$ and $z*$ to obtain a prefix code for S

Algorithm

To construct a prefix code for an alphabet S, with given frequencies:

If S has two letters then

Encode one letter using 0 and the other letter using 1

Else

Let $y*$ and $z*$ be the two lowest-frequency letters

Form a new alphabet S' by deleting $y*$ and $z*$ and replacing them with a new letter ω of frequency $f_{y*} + f_{z*}$

Recursively construct a prefix code γ' for S', with tree T'

Define a prefix code for S as follows:

Start with T'

Take the leaf labeled ω and add two children below it labeled $y*$ and $z*$

Endif

Example

First let's consider the behavior of the algorithm on our sample instance with S = {a, b, c, d, e} and frequencies:

$f_a = .32$, $f_b = .25$, $f_c = .20$, $f_d = .18$, $f_e = .05$

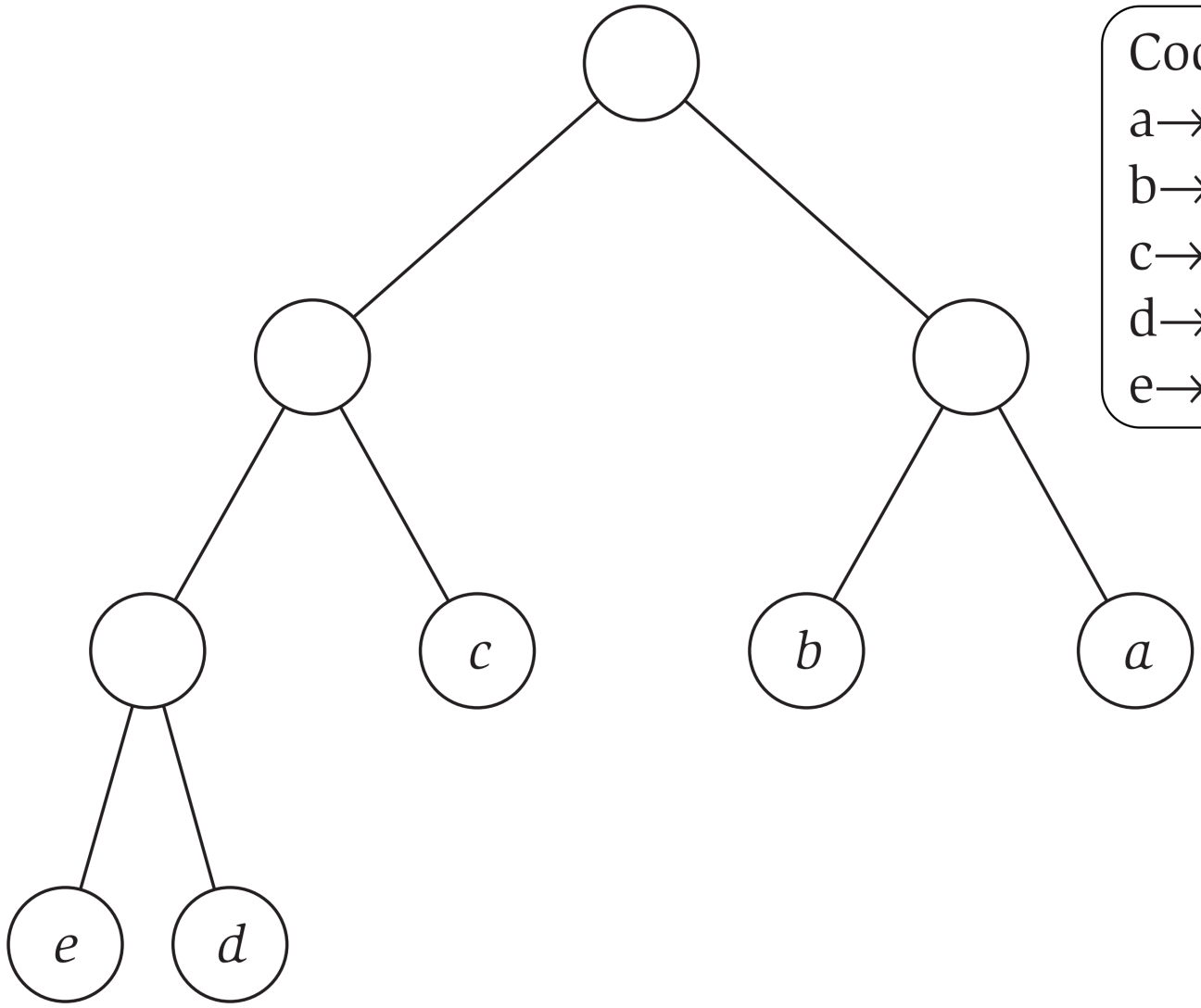
The algorithm would first merge d and e into a single letter—let's denote it (de)—of frequency .18 + .05 = .23. We now have an instance of the problem on the four letters S' = {a, b, c, (de)}

The two lowest-frequency letters in S' are c and (de), so in the next step we merge these into the single letter (cde) of frequency .20 + .23 = .43

Example

This gives us the three-letter alphabet {a, b, (cde)}.

Next we merge a and b, and this gives us a two-letter alphabet, at which point we invoke the base case of the recursion



Bellman-Ford

1. Initialize distance[source] = 0 and distance[v] = ∞ for all vertices v ≠ source.
2. For i = 1 to |V| - 1:
   a. For each edge (u, v) with weight w:
      i. If distance[u] + w < distance[v], then distance[v] = distance[u] + w.
3. For each edge (u, v) with weight w:
   a. If distance[u] + w < distance[v], then report a negative weight cycle.
4. Return the distance array.

Dynamic Programming

1. Initialize dist[0][s] = 0 and dist[0][v] = ∞ for all vertices v ≠ s.
2. For k = 1 to |V| - 1:
   a. For each edge (u, v) with weight w:
      i. dist[k][v] = min(dist[k][v], dist[k-1][u] + w(u, v)).
3. For each edge (u, v) with weight w:
   a. If dist[|V| - 1][u] + w(u, v) < dist[|V| - 1][v], then report a negative weight cycle.
4. Return dist[|V| - 1][v] for all vertices v.